DEVOPS ASSESSMENT

Implementation report | Todo Summary Assistant | 17 October 2026

Executive summary

Repository-side implementation is substantially prepared. The backend tests, frontend production build, and local application functionality were verified. A full container-to-container deployment did not run in this Codespaces environment because Docker bridge traffic between containers timed out. A documented local fallback was used instead: MySQL in Docker, with the Spring Boot backend and React frontend running directly in the Codespace.

AWS deployment was not performed. The project owner reported that the AWS account is suspended / AWS access is unavailable. No EC2, RDS, ECR, IAM, security-group, or cloud deployment was created or tested. This report records that as the stated blocker; it is not an independent AWS account status check.

Assessment snapshot

Area	Status	Evidence / boundary
Repository implementation	Prepared	Dockerfiles, workflow, monitoring config, architecture diagram, README and operations documents are present.
Local application	Verified via Codespaces fallback	MySQL healthy; backend health UP; frontend HTTP 200; todo create and delete verified through frontend proxy.
Full Docker Compose stack	Not verified here	Images built and Compose config parsed, but the Codespaces Docker bridge timed out between containers.
AWS deployment	Not done	Blocked by owner-reported suspended/unavailable AWS account access; setup instructions are provided.
Estimated completion	~65% overall verified	Repository work is prepared, but the required AWS deployment and full cloud pipeline remain unverified.

What “complete” means in this report

Configuration and documentation are distinguished from running infrastructure. A checked-in workflow is not described as a successful GitHub Actions deployment; metrics configuration is not described as live AWS monitoring; and the local fallback is not described as a production AWS release.

Assessment requirements: what was asked and delivered

The table below maps the assessment rubric to repository deliverables and the verification actually performed.

Rubric	Asked by assessment	Prepared / done	Verification status
4.1 Local run	Document runtime needs, variables, assumptions, local run steps.	README updated; environment variables documented; Codespaces fallback included.	Verified local MySQL + backend + frontend; UI/API and CRUD exercised.
4.2 Docker	Multi-stage backend/frontend images, non-root, external config; Compose optional.	Backend and frontend Dockerfiles, ignore files, Nginx proxy, Compose stack.	Both images built; Compose config validated. Full bridge networking blocked in this Codespace.
4.3 CI/CD	Push/PR test, build/tag/publish, deploy, health check.	GitHub Actions workflow: Maven tests, frontend build, SHA images, ECR, OIDC + SSM deployment and health checks.	Workflow authored, not run in GitHub; needs AWS configuration.
4.4 AWS	EC2 + private MySQL RDS, IAM, restricted security groups, diagram.	AWS setup/security guidance, deployment script, PNG architecture diagram.	Not provisioned or deployed because AWS account access is reported unavailable/suspended.
4.5 Monitoring	Prometheus application/host scraping, Grafana dashboard, at least two alerts.	Actuator/Prometheus endpoint; Prometheus rules; dashboard panels; Node Exporter/cAdvisor in Compose.	Endpoint and rules validated; Grafana dashboard loaded locally. Bridge restrictions prevent reliable scrape targets here.
4.6 Failure/ rollback	Written answers for six failure scenarios.	FAILURE_AND_ROLLBACK.md.	Document present; no real cloud rollback exercise (AWS unavailable).
4.7 Operations	Metrics, logs, alert noise, early detection.	MONITORING_AND_OPERATIONS.md.	Document and alert definitions present; live production alert routing not configured.

Security and application enablement

No real credentials were added to source. Runtime settings use environment variables / a planned Secrets Manager secret. The EC2 deployment design uses GitHub OIDC and an instance role rather than long-lived AWS keys. The app changes were limited to DevOps enablement: Actuator metrics, an H2-only test profile, configurable CORS/API URL, and a React development proxy. Cohere and Slack credentials remain optional for CRUD.

Monitoring key clarification

Prometheus and Grafana require no external API keys for this local setup. Grafana uses a local admin password configured in the ignored .env file. Cohere and Slack keys are only needed to demonstrate those optional integrations. AWS credentials/resources are needed only for cloud publishing and deployment.

Validation evidence and remaining work

Commands/results recorded during implementation

Backend: `mvn --batch-mode --no-transfer-progress test` — **BUILD SUCCESS**, 2 tests, 0 failures/errors. Tests use in-memory H2 and do not claim MySQL integration coverage.

Frontend: `CI=true npm run build` — compiled successfully. `CI=true npm test -- --watchAll=false --passWithNoTests` — exited successfully; repository currently defines no frontend tests.

Running fallback: health endpoint returned `{"status":"UP"}`; frontend responded HTTP 200; test todo POST returned HTTP 201 and DELETE returned HTTP 204 via the React proxy. Temporary test todos were deleted.

Monitoring/config: Prometheus configuration and four alert rules passed `promtool` checks; Grafana dashboard JSON validated and provisioned. Prometheus and Grafana HTTP health endpoints returned 200 in the earlier local run. Scrape targets requiring bridge networking were down in this Codespaces environment.

Captured terminal evidence (not an AWS-console screenshot)

This is a transcription of actual command results from the local run, not a fabricated screenshot. AWS account status below is owner-reported; no AWS console evidence was available.

```
GET /actuator/health -> HTTP 200 {"status":"UP"}
GET frontend / -> HTTP 200
POST /api/todos -> HTTP 201
DELETE /api/todos/{id}-> HTTP 204
Docker MySQL -> healthy (loopback port 127.0.0.1:3306)
```

Why AWS was not completed

The owner stated AWS access is unavailable because the account is suspended. The AWS portion requires an accessible account to create/configure EC2, private RDS, ECR, IAM/OIDC, security groups, Secrets Manager and a real deployment. These changes could not be performed or honestly claimed as completed without account access. The repository contains a reproducible plan and pipeline configuration ready for the owner to configure and validate after AWS access is restored.

Remaining checklist

Priority	Next action
1	Restore/arrange AWS account access; provision the resources and IAM/network setup in aws/aws-setup.md.
2	Configure GitHub OIDC role, repository variables/secrets, and Secrets Manager values without putting secrets in source.
3	Run the GitHub Actions workflow on main; verify ECR image publication, SSM deployment, app health, and rollback behavior.
4	Run the full Compose stack on a Docker host with working bridge networking, then confirm all Prometheus targets are UP and test real alert delivery.
5	Commit/push the prepared files to the requested submission repository and provide the repository link or ZIP.

Repository status at report generation: changes are in the Codespace working tree and have not been committed or pushed. No AWS deployment was attempted.